```cpp
#include <iostream>
#include <cmath>
#include <vector>
#include <algorithm>
//1. 核心精度层与二维基础 (Precision & 2D Primitives)
using namespace std;

const double eps = 1e-8;
const double PI = acos(-1.0);

// 核心符号判定
inline int sgn(double x) {
    if (abs(x) < eps) return 0;
    return x < 0 ? -1 : 1;
}

struct Point {
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    Point operator+(const Point &b) const { return Point(x + b.x, y + b.y); }
    Point operator-(const Point &b) const { return Point(x - b.x, y - b.y); }
    Point operator*(double k) const { return Point(x * k, y * k); }
    Point operator/(double k) const { return Point(x / k, y / k); }
    bool operator==(const Point &b) const { return sgn(x - b.x) == 0 && sgn(y - b.y) == 0; }
    // 字典序比较：优先 x，其次 y
    bool operator<(const Point &b) const { return sgn(x - b.x) == 0 ? sgn(y - b.y) < 0 : x < b.x
};
typedef Point Vector;

// 核心算子
inline double Cross(const Vector &a, const Vector &b) { return a.x * b.y - a.y * b.x; }
inline double Dot(const Vector &a, const Vector &b) { return a.x * b.x + a.y * b.y; }
inline double Length(const Vector &a) { return sqrt(Dot(a, a)); }
inline double Length2(const Vector &a) { return Dot(a, a); }

// 向量逆时针旋转 rad 弧度
Vector Rotate(const Vector &a, double rad) {
    return Vector(a.x * cos(rad) - a.y * sin(rad), a.x * sin(rad) + a.y * cos(rad));
}

// 整数几何特化版 (当题目保证坐标为整数，极大降低 WA 概率)
// typedef long long ll;
```

```
// struct PointInt {
//     ll x, y;
//     PointInt(ll x=0, ll y=0): x(x), y(y) {}
//     PointInt operator-(const PointInt &b) const { return PointInt(x - b.x, y - b.y); }
//     bool operator<(const PointInt &b) const { return x < b.x || (x == b.x && y < b.y); }
// };
// inline ll CrossInt(const PointInt &a, const PointInt &b) { return a.x * b.y - a.y * b.x; }
//2. 直线与线段 (Lines & Segments)
struct Line {
    Point p, v; // p 为直线上一点，v 为方向向量
    double ang; // 极角，半平面交备用
    Line() {}
    Line(Point p, Point v) : p(p), v(v) { ang = atan2(v.y, v.x); }
    bool operator<(const Line &b) const {
        if (sgn(ang - b.ang) != 0) return ang < b.ang;
        return sgn(Cross(b.v, p - b.p)) < 0; // 极角相同，靠左的排前
    }
};

// 判断点 p 是否在线段 a-b 上
bool OnSegment(const Point &p, const Point &a, const Point &b) {
    return sgn(Cross(a - p, b - p)) == 0 && sgn(Dot(a - p, b - p)) <= 0;
}

// 两直线交点 (调用前需保证不平行: sgn(Cross(a.v, b.v)) != 0)
Point GetLineIntersection(const Line &a, const Line &b) {
    Point u = a.p - b.p;
    double t = Cross(b.v, u) / Cross(a.v, b.v);
    return a.p + a.v * t;
}

// 点 p 到直线 L 的距离
double DistancePointToLine(const Point &p, const Line &L) {
    return abs(Cross(L.v, p - L.p)) / Length(L.v);
}

// 点 p 到直线 L 的投影点
Point GetLineProjection(const Point &p, const Line &L) {
    double t = Dot(L.v, p - L.p) / Length2(L.v);
    return L.p + L.v * t;
}
//3. 圆的基础操作 (Circles)
struct Circle {
```

```
    Point c;
    double r;
    Circle(Point c = Point(), double r = 0) : c(c), r(r) {}
};

// 求两圆交点，返回交点个数 (-1: 重合, 0: 相离/内含, 1: 相切, 2: 相交)
int CircleIntersection(Circle C1, Circle C2, Point &p1, Point &p2) {
    double d = Length(C1.c - C2.c);
    if (sgn(d) == 0) return sgn(C1.r - C2.r) == 0 ? -1 : 0;
    if (sgn(d - (C1.r + C2.r)) > 0 || sgn(d - abs(C1.r - C2.r)) < 0) return 0;

    double a = atan2(C2.c.y - C1.c.y, C2.c.x - C1.c.x);
    double cos_val = (C1.r * C1.r + d * d - C2.r * C2.r) / (2 * C1.r * d);
    cos_val = max(-1.0, min(1.0, cos_val)); // 防止 acos(1.00000001) 产生 NaN
    double da = acos(cos_val);

    p1 = Point(C1.c.x + C1.r * cos(a - da), C1.c.y + C1.r * sin(a - da));
    p2 = Point(C1.c.x + C1.r * cos(a + da), C1.c.y + C1.r * sin(a + da));
    return sgn(da) == 0 ? 1 : 2;
}

// 求三点外接圆 (要求三点不共线)
Circle GetCircumcircle(Point p1, Point p2, Point p3) {
    Point mid1 = (p1 + p2) / 2.0;
    Point v1 = Point(-(p2.y - p1.y), p2.x - p1.x);
    Point mid2 = (p1 + p3) / 2.0;
    Point v2 = Point(-(p3.y - p1.y), p3.x - p1.x);
    Point c = GetLineIntersection(Line(mid1, v1), Line(mid2, v2));
    return Circle(c, Length(c - p1));
}
//4. 二维凸包 (Andrew's Algorithm)
// 求凸包 (去除共线点，若需保留改 <= 为 <)
int Andrew(Point* p, int n, Point* ch) {
    sort(p, p+n);
    int m=0;
    for(int i=0; i<n; ++i) {
        while(m>1 && sgn(Cross(ch[m-1]-ch[m-2], p[i]-ch[m-2]))<=0) m--;
        ch[m++] = p[i];
    }
    int k=m;
    for(int i=n-2; i>=0; --i) {
        while(m>k && sgn(Cross(ch[m-1]-ch[m-2], p[i]-ch[m-2]))<=0) m--;
        ch[m++] = p[i];
```

```
    }
    if(n>1) m--;
    return m;
}
//5. 旋转卡壳 (Rotating Calipers)
// 旋转卡壳求多边形直径 (返回最大距离)
double RotatingCalipers(Point* ch, int m) {
    if(m==1) return 0;
    if(m==2) return Length(ch[0]-ch[1]);
    double ans=0;
    for(int i=0, j=2; i<m; ++i) {
        // 当下一个点构成的三角形面积更大时，指针 j 逆时针移动
        while(Cross(ch[(i+1)%m]-ch[i], ch[(j+1)%m]-ch[i]) > Cross(ch[(i+1)%m]-ch[i], ch[j]-ch[i]
            j = (j+1)%m;
        ans = max({ans, Length(ch[i]-ch[j]), Length(ch[(i+1)%m]-ch[j])});
    }
    return ans;
}
//6. 半平面交 (Shamos-Hoey 改进版)
bool OnRight(const Line& L, const Point& p) {
    return sgn(Cross(L.v, p-L.p))<=0;
}

// 求半平面交，L 为有向直线数组，res 保存构成的凸多边形顶点，返回顶点数
int HalfPlaneIntersection(Line* L, int n, Point* res) {
    sort(L, L+n);
    int head=0, tail=0, m=1;
    // 剔除极角相同的直线，保留最内侧
    for(int i=1; i<n; ++i) if(sgn(L[i].ang-L[i-1].ang)!=0) L[m++] = L[i];
    vector<Line> q(m+1);
    vector<Point> p(m+1);
    q[0] = L[0];
    for(int i=1; i<m; ++i) {
        while(head<tail && OnRight(L[i], p[tail-1])) tail--;
        while(head<tail && OnRight(L[i], p[head])) head++;
        q[++tail] = L[i];
        if(sgn(Cross(q[tail].v, q[tail-1].v))==0) return 0; // 平行且反向，交集为空
        if(head<tail) p[tail-1] = GetLineIntersection(q[tail-1], q[tail]);
    }
    while(head<tail && OnRight(q[head], p[tail-1])) tail--;
    if(head<tail) p[tail] = GetLineIntersection(q[tail], q[head]);
    if(tail-head+1 < 3) return 0;
    int cnt=0;
```

```
    for(int i=head; i<=tail; ++i) res[cnt++] = p[i];
    return cnt;
}
//7. 最小圆覆盖 (Welzl's Algorithm)
// 最小圆覆盖 (需包含 <algorithm> 且确保 srand 初始化)
Circle MinimumEnclosingCircle(Point* p, int n) {
    random_shuffle(p, p+n);
    Circle C(p[0], 0);
    for(int i=1; i<n; ++i) {
        if(sgn(Length(C.c-p[i])-C.r)>0) {
            C = Circle(p[i], 0);
            for(int j=0; j<i; ++j) {
                if(sgn(Length(C.c-p[j])-C.r)>0) {
                    C.c = (p[i]+p[j])/2.0;
                    C.r = Length(p[i]-C.c);
                    for(int k=0; k<j; ++k) {
                        if(sgn(Length(C.c-p[k])-C.r)>0)
                            C = GetCircumcircle(p[i], p[j], p[k]);
                    }
                }
            }
        }
    }
    return C;
}
//8. 自适应辛普森积分 (Adaptive Simpson's Integration)
// F(x) 为根据题意自定的被积函数，如扫描线处各线段交集总长
extern double F(double x);

inline double simpson(double l,double r){
    double mid=(l+r)/2.0;
    return (F(l)+4.0*F(mid)+F(r))*(r-l)/6.0;
}

// 调用入口: asr(min_x, max_x, 1e-6, simpson(min_x, max_x), 20)
double asr(double l,double r,double eps,double ans,int step){
    double mid=(l+r)/2.0;
    double L=simpson(l,mid),R=simpson(mid,r);
    if(abs(L+R-ans)<=15.0*eps || step<=0) return L+R+(L+R-ans)/15.0;
    return asr(l,mid,eps/2.0,L,step-1)+asr(mid,r,eps/2.0,R,step-1);
}
//9. 三维基础结构与原语 (3D Primitives)
struct Point3{
```

```
    double x,y,z;
    Point3(double x=0,double y=0,double z=0):x(x),y(y),z(z){}
    Point3 operator+(const Point3 &b)const{return Point3(x+b.x,y+b.y,z+b.z);}
    Point3 operator-(const Point3 &b)const{return Point3(x-b.x,y-b.y,z-b.z);}
    Point3 operator*(double k)const{return Point3(x*k,y*k,z*k);}
    Point3 operator/(double k)const{return Point3(x/k,y/k,z/k);}
};
typedef Point3 Vector3;

inline double Dot(const Vector3 &a,const Vector3 &b){return a.x*b.x+a.y*b.y+a.z*b.z;}
inline Vector3 Cross(const Vector3 &a,const Vector3 &b){
    return Vector3(a.y*b.z-a.z*b.y,a.z*b.x-a.x*b.z,a.x*b.y-a.y*b.x);
}
inline double Length(const Vector3 &a){return sqrt(Dot(a,a));}

// 四面体体积 (混合积)
inline double VolumeTetrahedron(Point3 A,Point3 B,Point3 C,Point3 D){
    return abs(Dot(Cross(B-A,C-A),D-A))/6.0;
}
//10. Delaunay 空圆判定 (InCircle Predicate)
typedef long long ll;
typedef __int128_t lll; // 确保在 GCC 环境下编译

struct PointInt{
    ll x,y;
    PointInt(ll x=0,ll y=0):x(x),y(y){}
    PointInt operator-(const PointInt &b)const{return PointInt(x-b.x,y-b.y);}
};

// 判定 A, B, C 是否严格逆时针排列
inline bool CCW(PointInt A,PointInt B,PointInt C){
    return (B.x-A.x)*(C.y-A.y)-(B.y-A.y)*(C.x-A.x)>0;
}

// 断言：点 D 是否在 A, B, C 构成的外接圆内 (需保证 A,B,C 为逆时针)
bool InCircle(PointInt A,PointInt B,PointInt C,PointInt D){
    lll adx=A.x-D.x,ady=A.y-D.y;
    lll bdx=B.x-D.x,bdy=B.y-D.y;
    lll cdx=C.x-D.x,cdy=C.y-D.y;
    lll adz=adx*adx+ady*ady;
    lll bdz=bdx*bdx+bdy*bdy;
    lll cdz=cdx*cdx+cdy*cdy;
    lll det=adx*(bdy*cdz-bdz*cdy)-ady*(bdx*cdz-bdz*cdx)+adz*(bdx*cdy-bdy*cdx);
```

```
    return det>0;
}
```